

Lab 10 - Authentication & Authorization in ASP.NET Core MVC

What is Authentication and Authorization?

Authentication: Who are you?

Authorization: What are you allowed to do?

How ASP.NET Core Facilitates Authentication and Authorization:

ASP.NET Core makes Authentication and Authorization easy by providing:

- Built-in Identity system (`Microsoft.AspNetCore.Identity`) to manage users, passwords, roles, and claims.
- Integration with Entity Framework Core to store users securely in a database.
- Automatic cookie-based authentication — after login, a cookie is generated.
- Built-in `[Authorize]` and `[AllowAnonymous]` attributes to easily control access on controllers/actions.

Lets Begin :

Necessary Dependencies to Install

1. `Microsoft.AspNetCore.Identity.EntityFrameworkCore`
2. `Microsoft.EntityFrameworkCore.SqlServer`
3. `Microsoft.EntityFrameworkCore.Tools`
4. `Microsoft.AspNetCore.Authentication.Cookies`

1. Setup the Database Context

Create a Data folder → Add a new class called `ApplicationDbContext.cs`:

```
using Microsoft.EntityFrameworkCore;
```

```

using WebApplication3.Models.entities;

using Microsoft.AspNetCore.Identity.EntityFrameworkCore;

using Microsoft.AspNetCore.Identity;

namespace WebApplication3.Data

{

    public class ApplicationDbContext : IdentityDbContext

    {

        public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options) :
base(options)

        {

        }

        // Referencing the Student entity

        public DbSet<student> students { get; set; }

    }

}

```

2. Create the ViewModelsCreate the ViewModels

Inside **Models** folder, create two classes:

1. Create LoginViewModel.cs

```

using System.ComponentModel.DataAnnotations;

namespace WebApplication3.Models

{

    public class LoginViewModel

    {

    }

}

```

```

        [Required]

        [EmailAddress]

        public string Email { get; set; }

        [Required]

        [DataType(DataType.Password)]

        public string Password { get; set; }

        public bool RememberMe { get; set; }
    }
}

```

2. Create registerViewModel.cs

```

using System.ComponentModel.DataAnnotations;

namespace WebApplication3.Models
{
    public class RegisterViewModel
    {
        [Required]

        [EmailAddress]

        public string Email { get; set; }

        [Required]

        [DataType(DataType.Password)]

        public string Password { get; set; }

        [Required]

        [DataType(DataType.Password)]

```

```

        [Compare("Password", ErrorMessage = "Passwords do not match.")]

        public string ConfirmPassword { get; set; }

    }

}

```

3. Create Account Controller

Inside **Controllers** folder → Add a new Controller → name it **AccountController.cs**

```

using Microsoft.AspNetCore.Identity;

using Microsoft.AspNetCore.Mvc;

using WebApplication3.Models;

namespace WebApplication3.Controllers
{

    public class AccountController : Controller
    {

        private readonly UserManager<IdentityUser> _userManager;

        private readonly SignInManager<IdentityUser> _signInManager;


        public AccountController(UserManager<IdentityUser> userManager,
SignInManager<IdentityUser> signInManager)

        {

            _userManager = userManager;

            _signInManager = signInManager;

        }


        // GET: Account/Register

```

```
public IActionResult Register()
{
    return View();
}

// POST: Account/Register
[HttpPost]
public async Task<IActionResult> Register(RegisterViewModel model)
{
    if (ModelState.IsValid)
    {
        var user = new IdentityUser { UserName = model.Email, Email = model.Email };
        var result = await _userManager.CreateAsync(user, model.Password);

        if (result.Succeeded)
        {
            await _signInManager.SignInAsync(user, isPersistent: false);
            return RedirectToAction("Index", "Home");
        }

        foreach (var error in result.Errors)
        {
            ModelState.AddModelError("", error.Description);
        }
    }
}
```

```
        return View(model);
    }

    // GET: Account/Login
    public IActionResult Login()
    {
        return View();
    }

    // POST: Account/Login
    [HttpPost]
    public async Task<IActionResult> Login(LoginViewModel model)
    {
        if (ModelState.IsValid)
        {
            var result = await _signInManager.PasswordSignInAsync(model.Email,
model.Password, model.RememberMe, lockoutOnFailure: false);

            if (result.Succeeded)
            {
                return RedirectToAction("Index", "Home");
            }

            ModelState.AddModelError(string.Empty, "Invalid login attempt.");
        }

        return View(model);
    }

    // POST: Account/Logout
```

```

        [HttpPost]

        public async Task<IActionResult> Logout()
        {
            await _signInManager.SignOutAsync();

            return RedirectToAction("Index", "Home");
        }
    }
}

```

4. Create Views

Login.cshtml

```

@model WebApplication3.Models.LoginViewModel

<h2>Login</h2>

<form asp-action="Login" method="post">

    <div>

        <label asp-for="Email"></label>

        <input asp-for="Email" />

    </div>

    <div>

        <label asp-for="Password"></label>

        <input asp-for="Password" type="password" />

    </div>

    <div>

        <label asp-for="RememberMe"></label>

        <input asp-for="RememberMe" type="checkbox" />

```

```
</div>
```

```
<button type="submit">Login</button>
```

```
</form>
```

```
<a href="/Account/Register">Don't have an account? Register</a>
```

2. Register.cshtml

```
@model WebApplication3.Models.RegisterViewModel
```

```
<h2>Register</h2>
```

```
<form asp-action="Register" method="post">
```

```
<div>
```

```
<label asp-for="Email"></label>
```

```
<input asp-for="Email" />
```

```
</div>
```

```
<div>
```

```
<label asp-for="Password"></label>
```

```
<input asp-for="Password" type="password" />
```

```
</div>
```

```
<div>
```

```
<label asp-for="ConfirmPassword"></label>
```

```
<input asp-for="ConfirmPassword" type="password" />
```

```
</div>
```

```
<button type="submit">Register</button>
```

```
</form>
```

```
<a href="/Account/Login">Already have an account? Login</a>
```


5. Update Layout for Login/Logout Links

In your `_Layout.cshtml`, inside the `<ul class="navbar-nav">`, replace with

```
@using Microsoft.AspNetCore.Identity

@inject SignInManager<IdentityUser> SignInManager
@inject UserManager<IdentityUser> UserManager

<ul class="navbar-nav">

    @if (SignInManager.IsSignedIn(User))
    {
        <li class="nav-item">

            <a class="nav-link text-dark" asp-area="" asp-controller="Account"
            asp-action="Manage" title="Manage">Hello @User.Identity?.Name!</a>

        </li>

        <li class="nav-item">

            <form class="form-inline" asp-area="" asp-controller="Account"
            asp-action="Logout" method="post">

                <button type="submit" class="nav-link btn btn-link text-dark">Logout</button>

            </form>

        </li>
    }
    else
    {
        <li class="nav-item">

            <a class="nav-link text-dark" asp-controller="Account"
            asp-action="Login">Login</a>
```

```
        </li>

        <li class="nav-item">

            <a class="nav-link text-dark" asp-controller="Account"
            asp-action="Register">Register</a>

        </li>

    }

</ul>
```

6. Migrations

Add-Migration InitialIdentity

Update-Database

This will create the necessary tables like:

- AspNetUsers
- AspNetRoles
- AspNetUserRoles
- AspNetUserClaims

How It Works - Authentication and Authorization

- **Register:** Creates a user in `AspNetUsers`.
- **Login:** Verifies user, creates an authentication cookie.
- **Logout:** Deletes cookie, signs user out.
- **Authorization:** Can restrict pages using `[Authorize]` attribute.

Example of securing a Controller:

`[Authorize]`

```
public class StudentController : Controller
{
    public IActionResult Index()
    {
        return View();
    }
}
```